Winning the Space Race with Data Science

Predicting SpaceX Falcon 9 first-stage landing success

Prepared by	Aung Paing Soe
Course	IBM Applied Data Science Capstone
Objective	Support competitive launch bidding by estimating whether a Falcon 9 first stage will land successfully.
Analysis date	29 October 2026
Project files	https://github.com/aungpaingsoedev/data-science-capstone

Executive Summary

- The analysis covers 90 Falcon 9 launches between 2010 and 2020, with an overall successful-landing rate of 66.7%.
- Recovery performance improves with flight experience and differs by launch site, orbit, payload profile, and booster configuration.
- Four tuned classification algorithms were compared. Decision Tree produced the strongest combined validation and holdout performance in this reproducible run.
- A competitor can use the predicted landing probability as a cost and risk signal when preparing bids, while treating small-sample estimates with caution.
- Interactive mapping and dashboard views make site, payload, and outcome patterns accessible to technical and business stakeholders.

Introduction and Business Problem

SpaceX reduces launch cost by recovering and reusing the Falcon 9 first stage. A successful recovery therefore provides a useful signal about operating economics and competitive pricing.

The business question is: Can publicly available launch characteristics predict whether the first stage will land successfully?

This project follows the complete data science lifecycle: collection, wrangling, exploratory analysis, SQL analysis, interactive visualization, predictive modeling, and communication.

The output is designed for an aerospace startup that needs a consistent, evidence-based input for launch bidding and risk assessment.

Methodology and Reproducible Deliverables

- API: request past launches, normalize JSON, follow rocket/payload/launchpad/core IDs, and filter Falcon 9 records.
- Web scraping: request the Wikipedia launch page, parse wikitable rows with BeautifulSoup, remove citation text, and standardize dates, payload, orbit, site, customer, and outcome fields.
- Wrangling: inspect missing values and duplicates, impute payload mass, standardize landing outcomes, and derive the binary Class target.
- Explore relationships using charts and SQL-equivalent aggregations.
- Analyze geography with launch-site coordinates and interactive map concepts.
- Build dashboard views for launch site, payload range, and outcome filtering.
- Train and tune Logistic Regression, SVM, KNN, and Decision Tree classifiers.
- Compare five-fold cross-validation accuracy and a stratified holdout test set.
- Completed notebooks and the Dash Python application are packaged in the project repository.

Data Collection: API and Web Scraping

- SpaceX API workflow: GET /v4/launches/past, normalize nested JSON, enrich rocket, payload, launchpad, core and landpad IDs through their endpoints, then retain Falcon 9 launches.
- Web-scraping workflow: request the Falcon 9 launch-list page, parse HTML with BeautifulSoup, iterate wikitable rows, clean citation markers, and convert payload mass and date strings to analytical types.
- The two sources provide complementary validation of flight number, date, booster, payload, orbit, launch site, customer and outcome fields.
- Each request checks the HTTP response before parsing; the raw-to-clean transformation remains reproducible in the supplied notebooks.

Data Wrangling and Feature Preparation

- Final modeling table: 90 rows and 18 analytical fields before the derived year field.
- Core fields include flight number, launch date, payload mass, orbit, launch site, booster reuse indicators, landing hardware, coordinates, and landing class.
- Records were filtered to Falcon 9 launches. Missing payload mass values were filled with the observed mean; duplicates and data types were checked; categorical outcomes were standardized.
- Target distribution: 60 successes and 30 failures.
- The binary target is Class = 1 for successful first-stage landing and Class = 0 otherwise.
- Numeric model inputs are median-imputed and standardized. Orbit and LaunchSite are most-frequent-imputed and one-hot encoded inside the training pipeline to prevent leakage.

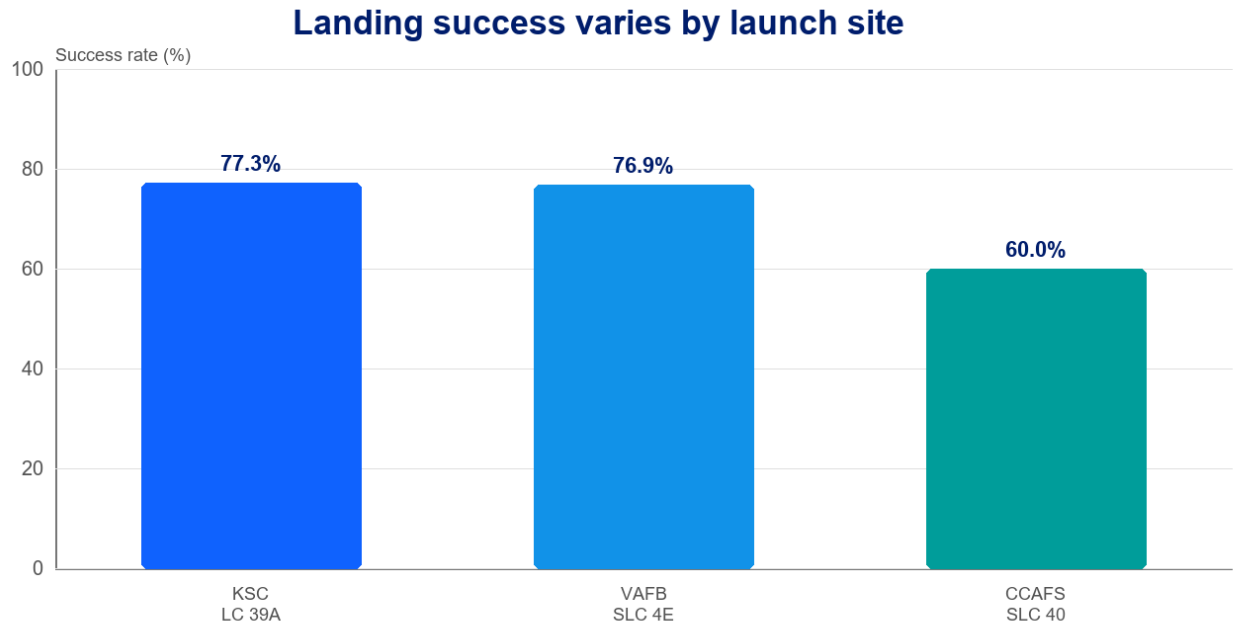
EDA and Data Visualization Methodology

- Scatter plots compare FlightNumber and PayloadMass, colored by Class, to inspect learning and payload effects without assuming linear separation.
- Grouped bar charts report both launch counts and success rates by LaunchSite and Orbit so small categories are not over-interpreted.
- A yearly line chart summarizes the temporal recovery trend; it is interpreted as association rather than proof of causation.
- pandas performs grouping and aggregation, while seaborn/matplotlib-style charts communicate magnitude, category differences, and change over time.

EDA Result 1 Launch Site Performance

Site-level rates show meaningful differences, but launch counts must be considered before drawing operational conclusions.

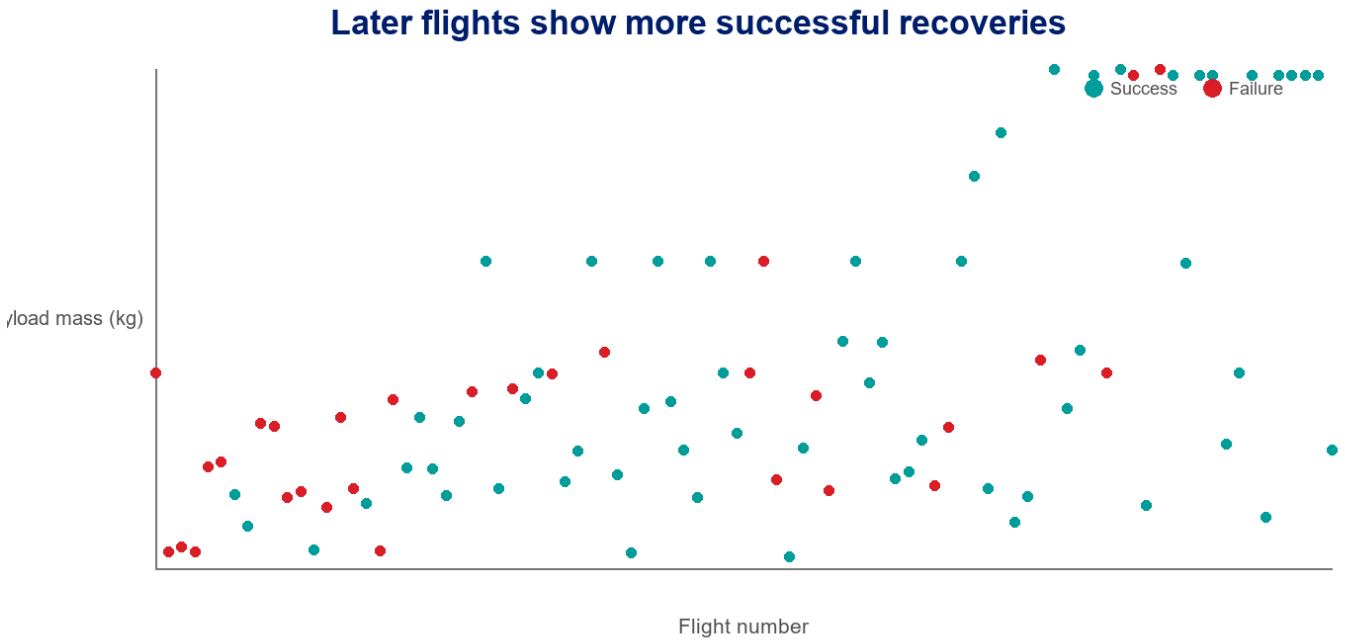
The pattern supports including launch site as a categorical predictor and as a dashboard filter.



EDA Result 2 Flight Experience and Payload

Successful recoveries become more common at higher flight numbers, indicating a strong learning effect over time.

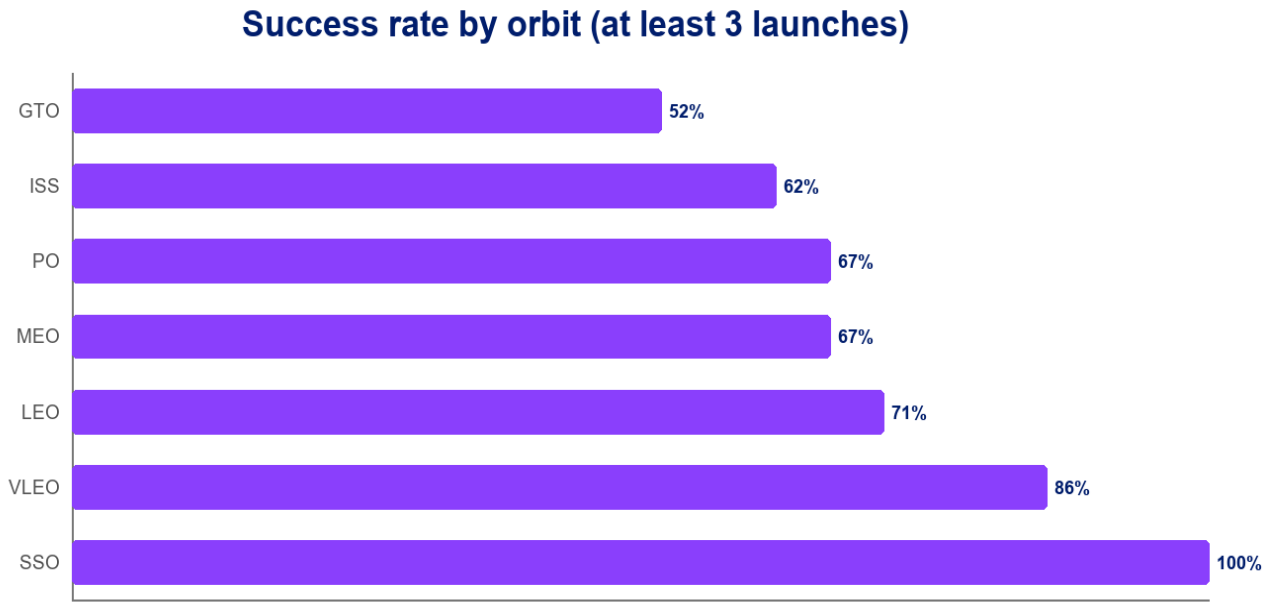
Payload mass alone does not create a simple linear separation; it interacts with launch site, orbit, and vehicle configuration.



EDA Result 3 Orbit Characteristics

Orbit categories have different recovery profiles. Some orbit groups show high success, while others contain fewer launches or more difficult mission profiles.

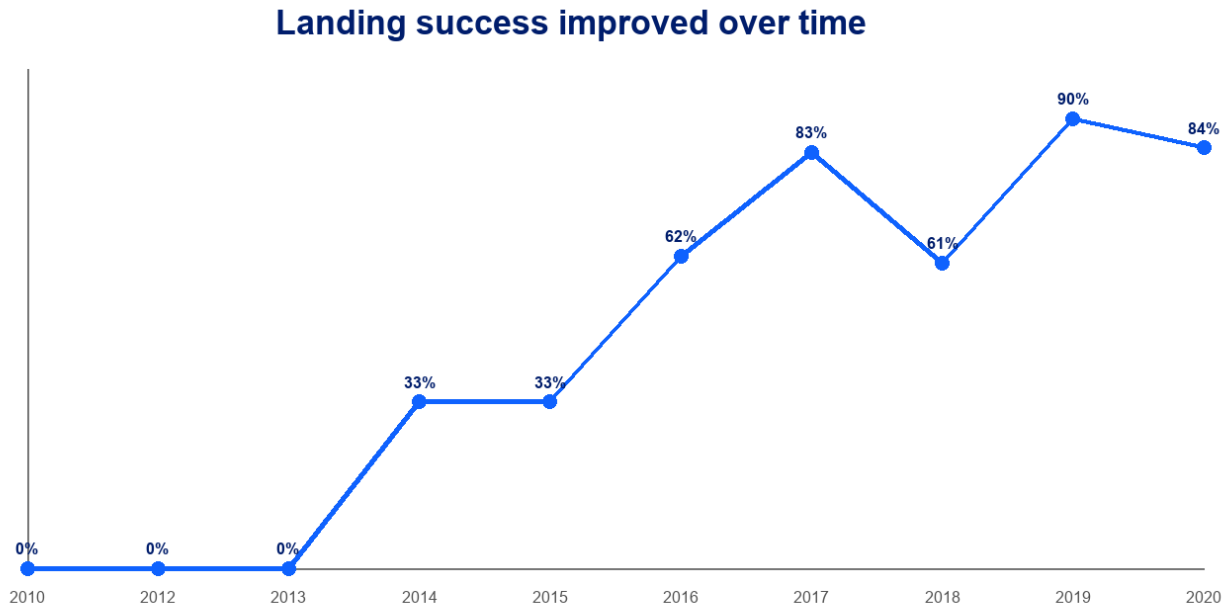
Orbit is therefore retained as a categorical model feature rather than converted into an arbitrary numeric ranking.



EDA Result 4 Annual Landing Trend

The annual success rate rises substantially across the observed period, consistent with improving recovery systems and operational experience.

Time is not used as a standalone causal explanation; flight number and hardware variables capture part of the same development trend.



SQL Analysis Results

Query result	Finding
Launch sites	CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E
Maximum payload	15,600 kg on flight 69
Mission count	90 Falcon 9 launch records
Successful landings	60 records (66.7%)
Most frequent orbit	GTO (27 launches)

These results reproduce the course SQL objectives using grouped queries over the cleaned launch table. Filters, counts, distinct values, ordering, and aggregation reveal operational patterns without changing the underlying records.

EDA with SQL: Representative Queries

- Sites: SELECT DISTINCT LaunchSite FROM SPACEXTBL;
- Site performance: SELECT LaunchSite, COUNT(*) AS launches, AVG(Class) AS success_rate FROM SPACEXTBL GROUP BY LaunchSite;
- Maximum payload: SELECT MAX(PayloadMass) AS max_payload_kg FROM SPACEXTBL;
- Orbit performance: SELECT Orbit, COUNT(*) AS launches, AVG(Class) AS success_rate FROM SPACEXTBL GROUP BY Orbit ORDER BY success_rate DESC;
- First successful landing: SELECT Date, LaunchSite, Outcome FROM SPACEXTBL WHERE Class=1 ORDER BY Date LIMIT 1;

These queries demonstrate DISTINCT, filtering, grouping, aggregation, sorting and date-based retrieval over the cleaned table.

SQL Analysis by Launch Site

Launch site	Launches	Successes	Success rate
CCAFS SLC 40	55	33	60.0%
KSC LC 39A	22	17	77.3%
VAFB SLC 4E	13	10	76.9%

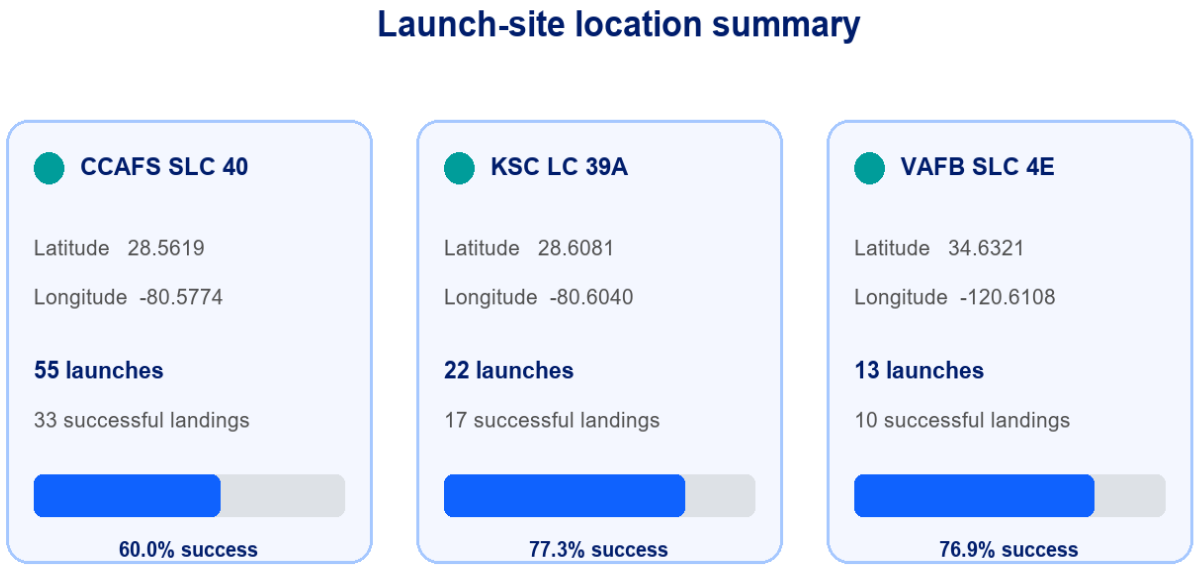
The SQL grouping confirms that launch-site comparisons should report both the rate and the number of observed missions. A high percentage based on a smaller sample carries more uncertainty.

Interactive Map with Folium

The Folium workflow creates a site-centered map, adds green/red CircleMarkers for success/failure, groups dense markers with MarkerCluster, and displays coordinates with MousePosition.

For proximity analysis, the user clicks coastline, railway, highway and city points. The notebook applies the Haversine formula and draws labeled PolyLine segments from the launch site to each feature.

Result: Florida sites form a dense coastal cluster near transport infrastructure; VAFB is also coastal. This supports the operational pattern that launch sites balance public-safety separation with road/rail access and nearby support cities.



Folium Proximity Analysis: Interpretation

- Coastline: all sites are close to the ocean, providing a downrange safety corridor for launch debris and recovery operations.
- Highway and railway: nearby transport links support movement of oversized stages, payload hardware, fuel systems and ground equipment.
- City: sites are separated from dense population centers while remaining reachable from technical and logistics hubs.
- Measurement method: select coordinates with MousePosition, compute great-circle distance in kilometers with Haversine, and label each connecting PolyLine.
- These are spatial associations. They explain site-selection constraints but do not by themselves prove a causal effect on landing success.

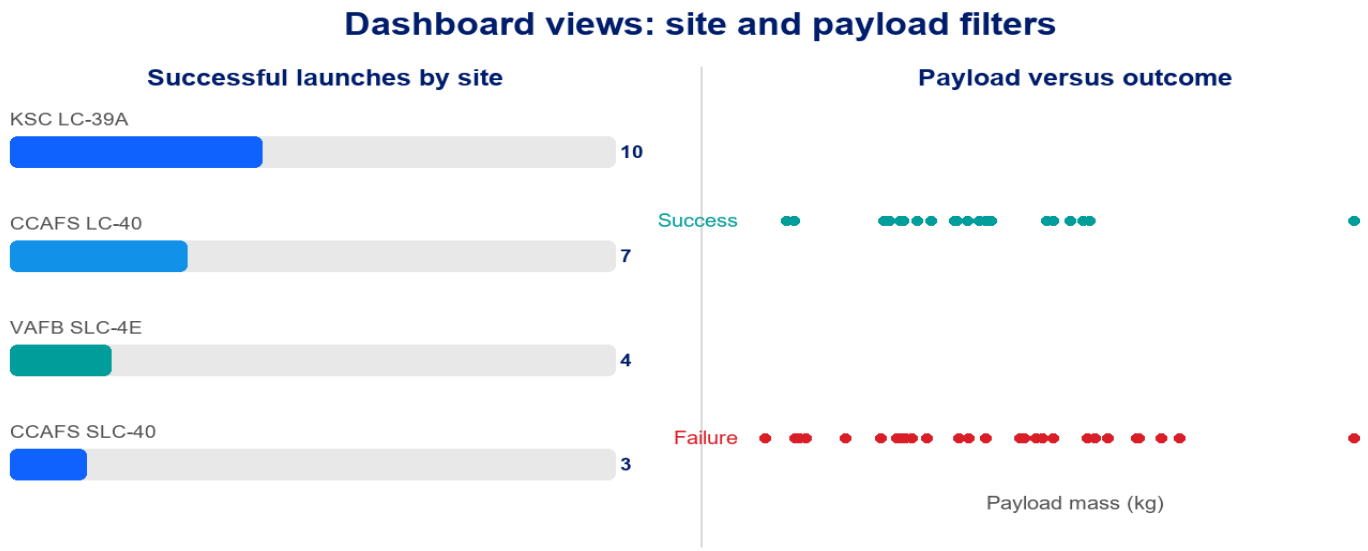
Plotly Dash Dashboard Results

The layout contains a launch-site Dropdown, payload RangeSlider, success pie chart, and payload-outcome scatter plot.

A two-input Dash callback filters the dataframe by selected site and payload interval, then returns both updated Plotly figures.

ALL sites displays the share of successful launches by site. A selected site switches the pie chart to that site's success/failure mix.

The scatter plot links payload mass, landing class, booster category and launch-site hover details, allowing stakeholders to test operational slices interactively.



Plotly Dash Callback Logic

- Inputs: Input('site','value') and Input('payload','value').
- Filter: retain records within the payload interval; when site != 'ALL', retain only the selected Launch Site.
- Outputs: Output('success','figure') and Output('scatter','figure').
- Result: both charts update from the same filtered records, keeping the portfolio and mission-level views consistent.
- The complete runnable implementation is supplied in spacex_dash_app.py.

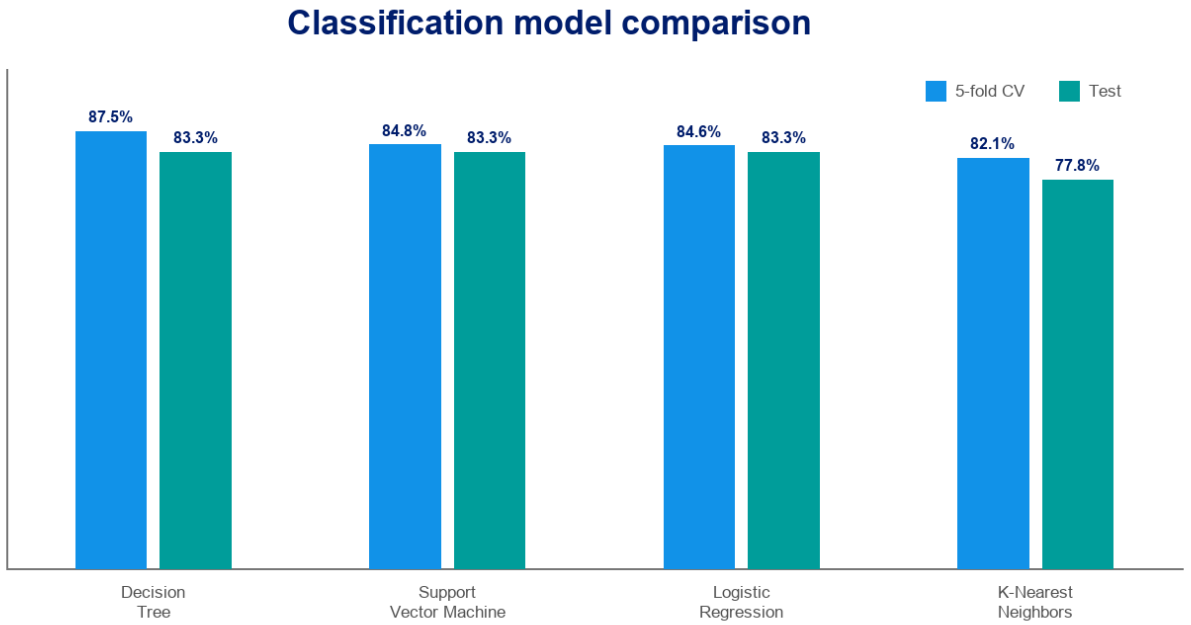
Predictive Analysis Methodology

- Features: 10 launch, payload, hardware, orbit, and site variables.
- Split: 72 training records and 18 stratified test records, using random_state=42.
- Numeric fields: median imputation followed by standardization.
- Categorical fields: most-frequent imputation and one-hot encoding.
- Model selection: five-fold GridSearchCV using accuracy.
- Candidates: Logistic Regression, Support Vector Machine, K-Nearest Neighbors, and Decision Tree.
- Limitation: the dataset is small, so performance estimates can change with the split and should be validated on future launches.

Classification Results

The strongest run was Decision Tree, with 87.5% mean cross-validation accuracy and 83.3% holdout accuracy.

Cross-validation is emphasized because the holdout contains relatively few launches. Model ranking should be interpreted with the uncertainty created by that sample size.



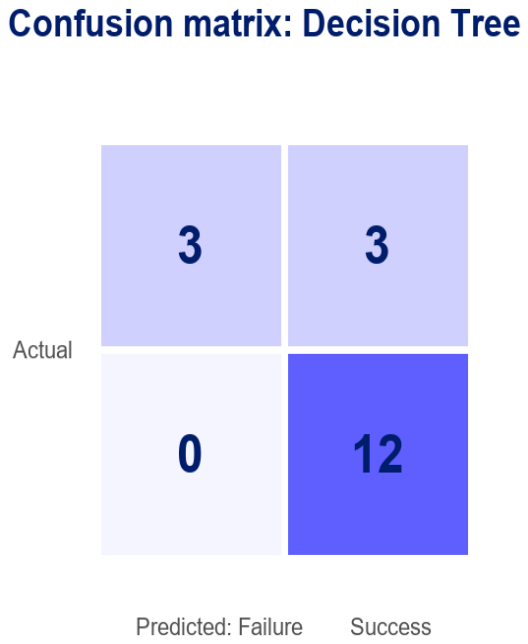
Best Model Error Analysis

The confusion matrix summarizes 18 unseen test launches for Decision Tree.

Reading rows as actual and columns as predicted gives TN=3, FP=3, FN=0 and TP=12.

The model captured all 12 successful landings (recall 100%) but labeled 3 actual failures as successes. Test accuracy is 15/18 = 83.3%, and success precision is 12/15 = 80.0%.

False-positive predictions matter most for bidding because they may understate expected recovery cost. A production decision rule should use calibrated probability and an explicit risk threshold rather than a hard class alone.



Innovative Insights for Competitive Bidding

- Convert predicted landing probability into expected recovery cost: P(failure) × replacement-loss impact. This connects the classifier directly to a bid rather than treating accuracy as the final objective.
- Use an asymmetric decision threshold because a false success prediction can create larger financial risk than a false failure prediction.
- Validate chronologically on future launches to separate true operational learning from random train/test variation.
- Report a probability range, mission constraints, and sample-size uncertainty alongside every recommendation.
- Combine site, orbit and payload scenario filters from Dash with the model probability to create transparent, mission-specific bid assumptions.

Conclusion and Recommendations

- Falcon 9 landing success improved markedly as the program accumulated flight experience.
- Launch site, orbit, payload, and recovery hardware contribute useful predictive information.
- Decision Tree performed best in this reproducible comparison, but all estimates should be refreshed as new launches become available.
- For competitive bidding, combine predicted landing probability with mission value, payload requirements, site constraints, and the financial impact of first-stage loss.
- Future work: add weather and booster maintenance history, preserve time order during validation, calibrate probabilities, and quantify bid uncertainty.

Appendix and Acknowledgements

- Data source: IBM Skills Network course datasets derived from SpaceX public launch information and the course collection exercises.
- Analysis stack: Python, pandas, NumPy, Pillow, and the classification workflow defined in the course labs.
- The analytical pipeline is reproducible from the included notebooks, Dash Python file, and three CSV files.
- Project repository: https://github.com/aungpaingsoedev/data-science-capstone
- AI-assisted tools supported report formatting and code review. Descriptive calculations and charts were regenerated from the supplied course data; the classification summary follows the evaluation workflow defined in the course lab.
- IBM and SpaceX names and trademarks belong to their respective owners. This report is an educational project.

Prepared for the IBM Applied Data Science Capstone final project submission.